SUPER 150 3rd YEAR TEST

TOTAL TIME → 2hrs + 10 mins (for the google form)

Below is the representation of the task which you need to perform.

-> lets say your roll number is 63
-> your set = (roll number) % 15 = 63%15 = 3
-> so your set number is 3
-> so you have to follow the below question set-3

```
QUESTION SET - 0
```

1. Create a schema-model according to the following diagrams (box shown below).
   → perform **Client-Side validation** as well (using bootstrap)
2. Perform **SHOW** & **DELETE** functionality. (use forms wisely if needed)
   → **'/songs/:id'** , '**/songs/:id/delete'** (use proper routing convention)

   # Form should have **inputs** according to the table below.
   # Only **songName** can be changed not the username.
3. Perform Authentication with passport. (following the schema structure)
   → '**/register'** , **'/login'** and show persons name on the top right next to login button.
4. Use *session* and *passport* to perform relevant tasks.

5. Follow proper folder structures
   → **Public** → style according to the template shown (just for reference)
   → **Views** → create proper template files
   → **Models** → make different schema
   → **Routes** → make different routing files

6. No need to have any dummy data.

7. Do not share the *node_modules* and *package-lock.json*.

User (model name)	**Song** (model name)
gender ✅ address ✅	songName ✅ duration ✅ singer ✅ timestamps ✅

userSchema (schema name)	**songSchema** (schema name)
gender: → type , mandatory **address**: → object **eg:** { **location** : “Delhi Paschim Vihar” }	**songName**: → type , mandatory , trim **duration**: → type , mandatory **singer**: → type , mandatory , trim **timestamps:** → created time

```
QUESTION SET - 1
```

1. Create a schema-model according to the following diagrams (box shown below).

2. Perform **CREATE** & **EDIT** functionality. (use forms wisely)
   → **‘/blogs’** , **‘/blogs/new’** , ‘**/blogs/:id/edit’** (use proper routing convention)

Form should have **inputs** according to the table below.
Only **blogContent** can be changed not the author name.

3. Perform Authentication with passport. (following the schema structure)
   → '**/register'** , **'/login'** and show persons name on the top right next to login button.
4. Use *session* and *passport* to perform relevant tasks.
5. Follow proper folder structures
   → **Public** → style according to the template shown (just for reference)
   → **Views** → create proper template files
   → **Models** → make different schema
   → **Routes** → make different routing files
6. No need to have any dummy data.
7. Do not share the *node_modules* and *package-lock.json*.

User (model name)	**Blog** (model name)
email ✅ address ✅	blogContent ✅ author ✅ timestamps ✅

userSchema (schema name)	**blogSchema** (schema name)
email: → type , mandatory , trim **address**: → objects **eg:** { **location** : "Delhi Paschim Vihar" }	**blogContent**: → type , mandatory , trim **author**: → type , mandatory , trim **timestamps:** → created time

```
QUESTION SET - 2
```

1. Create a schema-model according to the following diagrams (box shown below).
2. Perform **READ** & **DELETE** functionality. (use forms wisely if needed)
   → **'/tweets'** , '**/tweets/:id/delete'** (use proper routing convention)

   # Form should have **inputs** according to the table below.
   # Only **tweetContent** can be changed not the username.
3. Perform Authentication with passport. (following the schema structure)
   → '**/register'** , **'/login'** and show persons name on the top right next to login button.
4. Use *session* and *passport* to perform relevant tasks.
5. Follow proper folder structures
   → **Public** → style according to the template shown (just for reference)
   → **Views** → create proper template files
   → **Models** → make different schema
   → **Routes** → make different routing files
6. No need to have any dummy data.
7. Do not share the *node_modules* and *package-lock.json*.

User (model name)	**Tweet** (model name)
age ✅ address ✅	tweetContent ✅ username ✅ timestamps ✅

userSchema (schema name)	**tweetSchema** (schema name)
age: → type , mandatory **address**: → object **eg:** { **location** : "Delhi Paschim Vihar" }	**tweetContent**: → type , mandatory , trim **username**: → type , mandatory , trim **timestamps:** → created time

```
QUESTION SET - 3
```

1. Create a schema-model according to the following diagrams (box shown below).
2. Perform **SHOW** & **EDIT** functionality. (use forms wisely if needed)
   → **‘/products/:id’** , ‘**/products/:id/edit’** (use proper routing convention)

   # Form should have **inputs** according to the table below.
   # Only **productName** cannot be changed rest everything can change.
3. Perform Authentication with passport. (following the schema structure)
   → ‘**/register’** , **‘/login’** and show persons name on the top right next to login button.
4. Use *session* and *passport* to perform relevant tasks.
5. Follow proper folder structures
   → **Public** → style according to the template shown (just for reference)
   → **Views** → create proper template files
   → **Models** → make different schema
   → **Routes** → make different routing files
6. No need to have any dummy data.
7. Do not share the *node_modules* and *package-lock.json*.

User (model name)	**Product** (model name)
age ✅ phoneNumber ✅ gender ✅	productName ✅ price ✅ description ✅ timestamps ✅

userSchema (schema name)	**productSchema** (schema name)
age: → type , mandatory **phoneNumber**: → type **gender**: → type , mandatory	**productName**: → type , mandatory , trim **price**: → type , mandatory **description**: → type **timestamps:** → created time

QUESTION SET - 4

1. Create a schema-model according to the following diagrams (box shown below).
   → perform **Client-Side validation** as well (using bootstrap)
2. Perform **SHOW** & **DELETE** functionality. (use forms wisely if needed)
   → **'/guns/:id'** , '**/guns/:id/delete'** (use proper routing convention)

   # Form should have **inputs** according to the table below.
   # Only **gunName** cannot be changed rest everything can change.
3. Perform Authentication with passport. (following the schema structure)
   → '**/register'** , **'/login'** and show persons name on the top right next to login button.
4. Use *session* and *passport* to perform relevant tasks.
5. Follow proper folder structures
   → **Public** → style according to the template shown (just for reference)
   → **Views** → create proper template files
   → **Models** → make different schema
   → **Routes** → make different routing files
6. No need to have any dummy data.
7. Do not share the *node_modules* and *package-lock.json*.

User (model name)	**Gun** (model name)
age ✅ phoneNumber ✅ gender ✅ isLegal ✅	gunName ✅ price ✅ automatic ✅ timestamps ✅

userSchema (schema name)	**gunSchema** (schema name)

age: → type , mandatory **phoneNumber**: → type , mandatory **gender**: → type **isLegal**: → type , mandatory	**gunName**: → type , mandatory , trim **price**: → type , mandatory **automatic**: → type **timestamps:** → created time

QUESTION SET - 5

1. Create a schema-model according to the following diagrams (box shown below).
   → perform **Server-Side validation** as well (using JOI)
2. Perform **READ** & **CREATE** functionality. (use forms wisely if needed)
   → **'/phones/:id'** , **'/phones/new'** , '**/phones/:id/delete'** (use proper routing convention)

   # Form should have **inputs** according to the table below.
   # Everything can be changed.
3. Perform Authentication with passport. (following the schema structure)
   → '**/register'** , **'/login'** and show persons name on the top right next to login button.
4. Use *session* and *passport* to perform relevant tasks.
5. Follow proper folder structures
   → **Public** → style according to the template shown (just for reference)
   → **Views** → create proper template files
   → **Models** → make different schema
   → **Routes** → make different routing files
6. No need to have any dummy data.
7. Do not share the *node_modules* and *package-lock.json*.

User (model name)	**Phone** (model name)
phoneNumber ✅ gender ✅ monthlyIncome ✅	phoneName ✅ price ✅ isIOS ✅

userSchema (schema name)	**phoneSchema** (schema name)
phoneNumber: → type , mandatory **gender**: → type , mandatory **monthlyIncome**: → type	**phoneName**: → type , mandatory , trim **price**: → type , mandatory **isIOS**: → type

---

QUESTION SET - 6

1. Create a schema-model according to the following diagrams (box shown below).
   → perform **Client and Server Side validation** as well (using bootstrap and JOI)
2. Perform Authentication with passport. (following the schema structure)
   → '**/register'** , **'/login'** and show persons name on the top right next to login button.
3. Use *session* and *passport* to perform relevant tasks.
4. Follow proper folder structures
   → **Public** → style according to the template shown (just for reference)
   → **Views** → create proper template files
   → **Models** → make different schema
   → **Routes** → make different routing files
5. No need to have any dummy data.

6. Do not share the *node_modules* and *package-lock.json*.

User (model name)
age ✅ phoneNumber ✅ gender ✅ monthlyIncome ✅ address ✅

userSchema (schema name)
age: → type , mandatory **phoneNumber**: → type , mandatory **gender**: → type , mandatory **monthlyIncome**: → type **address**: → objects **eg:** { **location** : “delhi Paschim Vihar” }

QUESTION SET - 7

1. Create a schema-model according to the following diagrams (box shown below).
2. Perform **CREATE** , **SHOW** & **EDIT** functionality. (use forms wisely if needed)
   → **‘/scholarship/:id’ , ‘/scholarship/new’ , ‘/scholarship** , ‘**/scholarship/:id/edit’**

   (use proper routing convention)

   # Form should have **inputs** according to the table below.
   # Only **scholarshipHolder** cannot be changed rest everything can change.

3. Perform Authentication with passport. (following the schema structure)
   → '**/register'** , **'/login'** and show persons name on the top right next to login button.
4. Use *session* and *passport* to perform relevant tasks.
5. Follow proper folder structures
   → **Public** → style according to the template shown (just for reference)
   → **Views** → create proper template files
   → **Models** → make different schema
   → **Routes** → make different routing files
6. No need to have any dummy data.
7. Do not share the *node_modules* and *package-lock.json*.

User (model name)	**Scholarship** (model name)
age ✅ phoneNumber ✅ courseEnrolled ✅	scholarshipHolder ✅ percentage ✅ timestamps ✅

userSchema (schema name)	**scholarshipSchema** (schema name)
age: → type , mandatory **phoneNumber**: → type , mandatory **coursesEnrolled**: →objects **eg:** { **courseName** : "WEB" , }	**scholarshipHolder**: → type , mandatory , trim **percentage**: → type , mandatory **timestamps:** → created time

QUESTION SET - 8

1. Create a schema-model according to the following diagrams (box shown below).
   → perform **Client validation** as well (using bootstrap)
2. Perform **READ** & **DELETE** functionality. (use forms wisely if needed)
   → **'/canteen** , '**/canteen/:id/delete'** (use proper routing convention)

   # Form should have **inputs** according to the table below.
   # Only **FoodName** cannot be changed rest everything can change.
3. Perform Authentication with passport. (following the schema structure)
   → '**/register'** , **'/login'** and show persons name on the top right next to login button.
4. Use *session* and *passport* to perform relevant tasks.
5. Follow proper folder structures
   → **Public** → style according to the template shown (just for reference)
   → **Views** → create proper template files
   → **Models** → make different schema
   → **Routes** → make different routing files
6. No need to have any dummy data.
7. Do not share the *node_modules* and *package-lock.json*.

User (model name)	**Canteen** (model name)
isFaculty ✅ aadharNumber ✅ phoneNumber ✅ gender ✅	FoodName ✅ price ✅ image ✅ timestamps ✅

userSchema (schema name)	**canteenSchema** (schema name)
isFaculty: → type , mandatory **aadharNumber**: → type **phoneNumber**: → type **gender**: → type , mandatory	**FoodName**: → type , mandatory , trim **price**: → type , mandatory **image**: →type , mandatory **timestamps:** → created time

```
QUESTION SET - 9
```

1. Create a schema-model according to the following diagrams (box shown below).
2. Perform **READ** , **SHOW** & **DELETE** functionality. (use forms wisely if needed)
   → **‘/furniture/:id’ , ‘/furniture** , ‘**/furniture/:id/delete’** (use proper routing convention)

   # Form should have **inputs** according to the table below.
   # Only **furnitureName** cannot be changed rest everything can change.
3. Perform Authentication with passport. (following the schema structure)
   → ‘**/register’** , **‘/login’** and show persons name on the top right next to login button.
4. Use *session* and *passport* to perform relevant tasks.
5. Follow proper folder structures
   → **Public** → style according to the template shown (just for reference)
   → **Views** → create proper template files
   → **Models** → make different schema
   → **Routes** → make different routing files
6. No need to have any dummy data.
7. Do not share the *node_modules* and *package-lock.json*.

User (model name)	**Furniture** (model name)
isBuyer ✅ age ✅ phoneNumber ✅ favColor ✅	furnitureName ✅ price ✅ image ✅ timestamps ✅

userSchema (schema name)	**canteenSchema** (schema name)
isBuyer: → type , mandatory **age**: → type **phoneNumber**: → type **favColor**: → type	**furnitureName**: → type , mandatory , trim **price**: → type , mandatory **image**: →type , mandatory **timestamps:** → created time

QUESTION SET - 10

1. Create a schema-model according to the following diagrams (box shown below).
   → perform **Server side validation** as well (using JOI)
2. Perform **READ** , **EDIT** functionality. (use forms wisely if needed)
   → **'/subject/:id/edit' , '/subject** (use proper routing convention)

   # Form should have **inputs** according to the table below.
   # Only **subjectName** cannot be changed rest everything can change.
3. Perform Authentication with passport. (following the schema structure)
   → '**/register'** , **'/login'** and show persons name on the top right next to login button.
4. Use *session* and *passport* to perform relevant tasks.
5. Follow proper folder structures
   → **Public** → style according to the template shown (just for reference)
   → **Views** → create proper template files
   → **Models** → make different schema
   → **Routes** → make different routing files
6. No need to have any dummy data.
7. Do not share the *node_modules* and *package-lock.json*.

User (model name)	**Subject** (model name)
favSubject ✅ age ✅ collegeYear ✅ teacherName ✅	subjectName ✅ price ✅ image ✅ units ✅ timestamps ✅

userSchema (schema name)	**subjectSchema** (schema name)
favSubject: → type , mandatory **age**: → type **collegeYear**: → type	**subjectName**: → type , mandatory , trim **price**: → type , mandatory **image**: →type ,

teacherName: → type	mandatory **units**: →type , mandatory **timestamps:** → created time

QUESTION SET - 11

1. Create a schema-model according to the following diagrams (box shown below).
   → perform **Client side validation** as well (using bootstrap)
2. Perform **READ** , **EDIT , SHOW** functionality. (use forms wisely if needed)
   → **'/car/:id' , '/car/:id/edit' , '/car** (use proper routing convention)

   # Form should have **inputs** according to the table below.
   # Only **carName** cannot be changed rest everything can change.
3. Perform Authentication with passport. (following the schema structure)
   → '**/register'** , **'/login'** and show persons name on the top right next to login button.
4. Use *session* and *passport* to perform relevant tasks.
5. Follow proper folder structures
   → **Public** → style according to the template shown (just for reference)
   → **Views** → create proper template files
   → **Models** → make different schema
   → **Routes** → make different routing files
6. No need to have any dummy data.
7. Do not share the *node_modules* and *package-lock.json*.

User (model name)	**Car** (model name)
licence ✅ age ✅ owner ✅	carName ✅ price ✅ seater ✅ timestamps ✅

userSchema (schema name)	**carSchema** (schema name)
licence: → type , mandatory **age**: → type , mandatory **owner**: → type	**carName**: → type , mandatory , trim **price**: → type , mandatory **seater**: →type **timestamps:** → created time

QUESTION SET - 12

1. Create a schema-model according to the following diagrams (box shown below).
2. Perform **READ , SHOW , DELETE** functionality. (use forms wisely if needed)
   → **'/train/:id' , '/train/:id/delete' , '/train** (use proper routing convention)

   # Form should have **inputs** according to the table below.
   # Only **trainName** cannot be changed rest everything can change.
3. Perform Authentication with passport. (following the schema structure)
   → '**/register'** , **'/login'** and show persons name on the top right next to login button.
4. Use *session* and *passport* to perform relevant tasks.
5. Follow proper folder structures
   → **Public** → style according to the template shown (just for reference)
   → **Views** → create proper template files
   → **Models** → make different schema
   → **Routes** → make different routing files
6. No need to have any dummy data.
7. Do not share the *node_modules* and *package-lock.json*.

User (model name)	**Train** (model name)

age ✅ ticketNumber ✅ coaches ✅	trainName ✅ time ✅ seatsLeft✅ timestamps ✅

userSchema (schema name)	**trainSchema** (schema name)
age: → type , mandatory **ticketNumber**: → type , mandatory **coaches**: → type	**trainName**: → type , mandatory , trim **time**: → type , mandatory **seatsLeft**: →type **timestamps:** → created time

```
QUESTION SET - 13
```

1. Create a schema-model according to the following diagrams (box shown below).
   → perform **Server side validation** as well (using JOI)
2. Perform **READ , DELETE** functionality. (use forms wisely if needed)
   → **'/movie/:id/delete' , '/movie** (use proper routing convention)

   # Form should have **inputs** according to the table below.
   # Only **movieName** cannot be changed rest everything can change.
3. Perform Authentication with passport. (following the schema structure)
   → '**/register'** , **'/login'** and show persons name on the top right next to login button.
4. Use *session* and *passport* to perform relevant tasks.
5. Follow proper folder structures
   → **Public** → style according to the template shown (just for reference)
   → **Views** → create proper template files
   → **Models** → make different schema
   → **Routes** → make different routing files
6. No need to have any dummy data.
7. Do not share the *node_modules* and *package-lock.json*.

User (model name)	**Movie** (model name)
gender ✅ ticketNumber ✅ row ✅	movieName ✅ duration ✅ isAdult ✅ timestamps ✅

userSchema (schema name)	**movieSchema** (schema name)
gender: → type , mandatory **ticketNumber**: → type , mandatory **row**: → type	**movieName**: → type , mandatory , trim **duration**: → type , mandatory **isAdult**: →type **timestamps:** → created time

```
QUESTION SET - 14
```

1. Create a schema-model according to the following diagrams (box shown below).
   → perform **Client side validation** as well (using bootstrap)
2. Perform **SHOW , EDIT** functionality. (use forms wisely if needed)
   → **'/college/:id/edit' , '/college/:id** (use proper routing convention)

   # Form should have **inputs** according to the table below.
   # Only **collegeName** cannot be changed rest everything can change.
3. Perform Authentication with passport. (following the schema structure)
   → '**/register'** , **'/login'** and show persons name on the top right next to login button.
4. Use *session* and *passport* to perform relevant tasks.
5. Follow proper folder structures
   → **Public** → style according to the template shown (just for reference)
   → **Views** → create proper template files
   → **Models** → make different schema

→ **Routes** → make different routing files

6. No need to have any dummy data.
7. Do not share the *node_modules* and *package-lock.json*.

User (model name)	**College** (model name)
gender ✅ rollNumber ✅ year ✅	collegeName ✅ degreeName ✅ years ✅ timestamps ✅

userSchema (schema name)	**collegeSchema** (schema name)
gender: → type , mandatory **rollNumber**: → type , mandatory **year**: → type	**collegeName**: → type , mandatory , trim **degreeName**: → type , mandatory **years**: →type **timestamps:** → created time

```
QUESTION SET - 14
```

1. Create a schema-model according to the following diagrams (box shown below).
   → perform **Client side validation** as well (using bootstrap)
2. Perform **READ , EDIT** functionality. (use forms wisely if needed)
   → **'/standup/:id/edit' , '/standup** (use proper routing convention)

   # Form should have **inputs** according to the table below.
   # Only **comedianName** cannot be changed rest everything can change.
3. Perform Authentication with passport. (following the schema structure)
   → '**/register'** , **'/login'** and show persons name on the top right next to login button.

4. Use *session* and *passport* to perform relevant tasks.

5. Follow proper folder structures
   → **Public** → style according to the template shown (just for reference)
   → **Views** → create proper template files
   → **Models** → make different schema
   → **Routes** → make different routing files

6. No need to have any dummy data.

7. Do not share the *node_modules* and *package-lock.json*.

User (model name)	**Comedy** (model name)
gender ✅ isCouple ✅ passNumber ✅ age ✅	comedianName ✅ totalShows ✅ genre ✅ timestamps ✅

userSchema (schema name)	**comedySchema** (schema name)
gender: → type , mandatory **isCouple**: → type , mandatory **passNumber**: → type , mandatory **age**: → type	**comedianName**: → type , mandatory , trim **totalShows**: → type , mandatory **genre**: →type **timestamps:** → created time